SIE

Sistemas de Información Empresariales

odoo

Implementación

Construcción de módulos

¿Cuándo es necesario implementar?

- Necesidad no cubierta
- Módulo de pago excede el presupuesto
- Funcionalidad insuficiente
- Mejoras requeridas

Construye tu módulo

- Usaremos la formación “oficial” de Odoo

Getting started

Develop your own module with the Odoo framework. This step-by-step tutorial is crafted for newcomers and any other individual curious about Odoo development.

Beginner

Antes denominado “*Core training*”

- Tutorial paso a paso para **crear un módulo desde cero**
  Será un módulo para gestionar la venta de inmuebles (*Real Estate*)

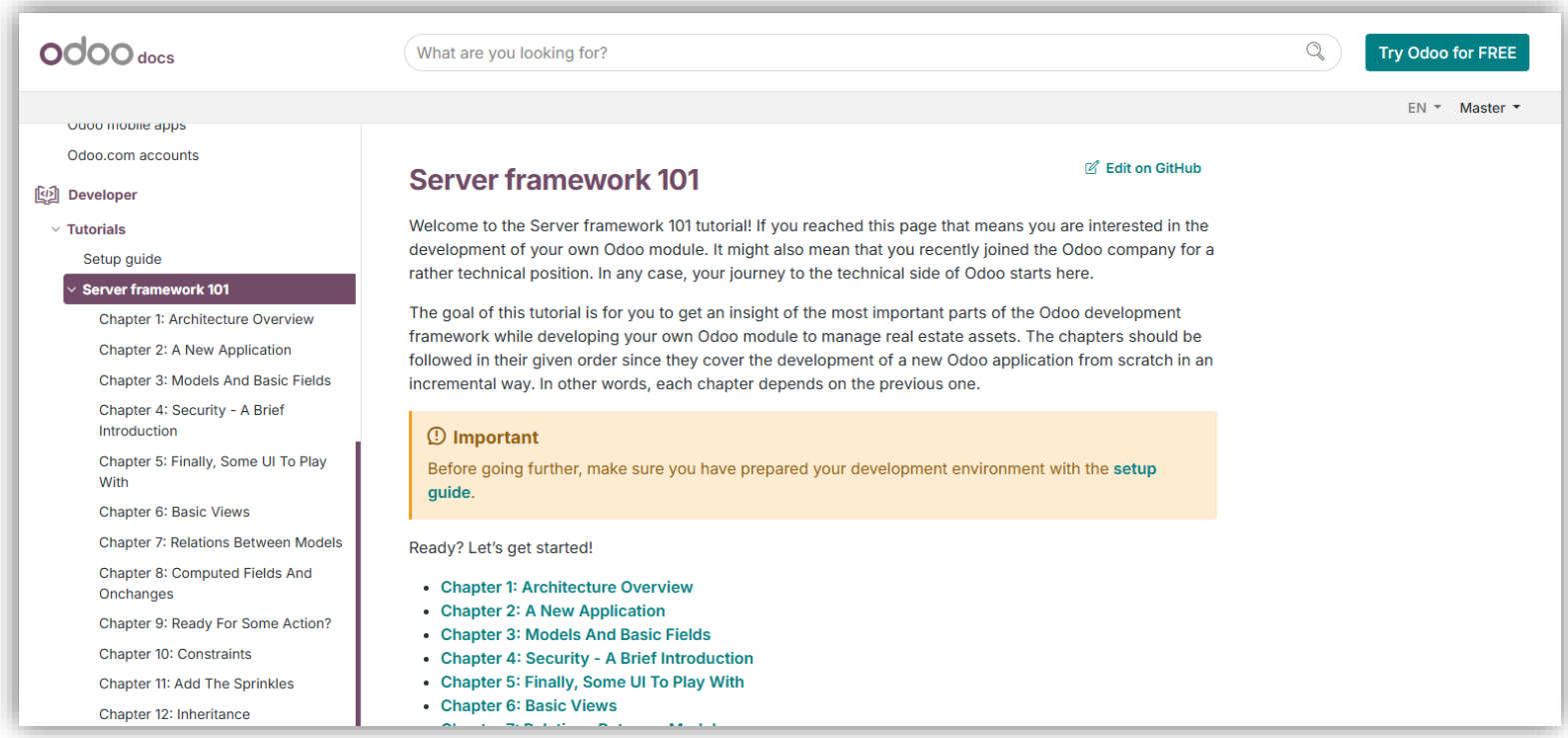
- Tutorial separado en varios **capítulos**
  - ✓ Cada capítulo introduce nuevos conceptos
  - ✓ Los capítulos deben completarse en orden
  - ✓ Se desarrolla el módulo de manera incremental
  - ✓ Un capítulo puede requerir modificar o eliminar lo hecho en un capítulo anterior

Chapter 1: Architecture Overview
Chapter 2: Development environment setup
Chapter 3: A New Application
Chapter 4: Models And Basic Fields
Chapter 5: Security - A Brief Introduction
Chapter 6: Finally, Some UI To Play With
Chapter 7: Basic Views
Chapter 8: Relations Between Models
Chapter 9: Computed Fields And Onchanges
Chapter 10: Ready For Some Action?
Chapter 11: Constraints
Chapter 12: Add The Sprinkles
Chapter 13: Inheritance
Chapter 14: Interact With Other Modules
Chapter 15: A Brief History Of QWeb
Chapter 16: The final word

Construye tu módulo

- Documentación

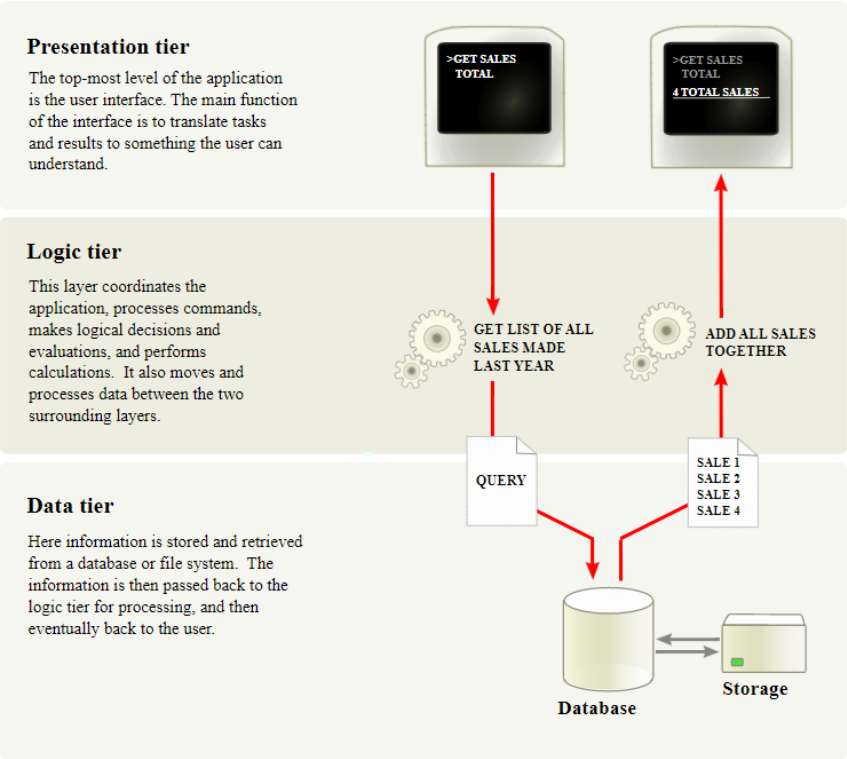
https://www.odoo.com/documentation/master/developer/tutorials/getting_started.html



Solución: https://github.com/odoo/technical-training-solutions (puede tener errores)

Capítulo 1 – Arquitectura

- Arquitectura multicapa
  - Presentación (HTML5, JS, CSS)
  - Lógica (Python)
  - Datos (PostgreSQL)



- Módulos (aka *addons*)
  - No todos son aplicaciones
  - <u>Ubicación</u>: *addons_path*
  - <u>Contenido</u> (todo opcional):
    - **Business objects** (Python class, fields → ORM → database columns), **Object views** (UI), **Data files** (XML or CSV: views or reports, configuration data, demo data,…), **Web controllers** and **Static web data**
  - <u>Estructura</u>: directorio con un fichero __manifest__.py como declaración e __init__.py si contiene Business objects (Python)

Capítulo 2 – Desarrollo

- Para un entorno de desarrollador necesitaremos:
  1. Obtener los ficheros de Odoo

     Opción a) *[más fácil]* Usar los ficheros de nuestra instalación de Odoo

     Opción b) Configurar Git y Github, clonar el repositorio de Odoo, hacer un fork,… e instalar las dependencias:
     - Python >= 3.7, pip, librerías,…
     - wkhtmltopdf
     - PostgreSQL, pgAdmin
  2. Configurar nuestro IDE (Visual Studio Code) para trabajar en la carpeta indicada por el parámetro addons_path

- Si usamos el instalador “oficial” podemos abrir en VSC la carpeta “server” de Odoo y ejecutar ‘odoo-bin’ desde el IDE (tal y como vimos antes)
- También podríamos trabajar solo con un simple editor de texto, pero sería bastante más “incómodo”

Capítulo 3 – Nueva Aplicación

- *Objetivo*: Crear un módulo vacío que podamos completar poco a poco para gestionar una **inmobiliaria**

- Crear el módulo ‘estate’:
  - ➢ Crear carpeta ‘estate’
  - ➢ Añadir `__init__.py` (vacío)
  - ➢ Añadir `__manifest__.py` (doc)
    - ✓ Añade los campos (‘name’,…)
    - ✓ Única dependencia: ‘base’
    - ✓ Añade: **`'application':True`** (ejemplo)

¡OJO! Son 4 ‘_’ y la extensión es ‘.py’ (comprueba que no haya ‘.txt’ oculto)

ejemplo

```
{
    'name': "A Module",
    'version': '1.0',
    'depends': ['base'],
    'author': "Author Name",
    'category': 'Category',
    'description': """
    Description text
    """,
    # data files always loaded at installation
    'data': [
        'views/mymodule_view.xml',
    ],
    # data files containing optionally loaded demonstration data
    'demo': [
        'demo/demo_data.xml',
    ],
}
```

Recuerda:
- Añade el directorio **padre** de ‘estate’ a *addons_path* (admite varias rutas separadas por comas y admite espacios)
- Modifica el ejemplo: añade lo que falta y borra lo que sobra (p.e. elimina de ‘data’ y ‘demo’ los ficheros que no existan)

Capítulo 3 – Nueva Aplicación

El fichero principal de Odoo (odoo o odoo-bin) incluye un parámetro opcional "**scaffold**":

¡OJO! NO lo necesitamos para el tutorial, ya que crearemos la estructura desde cero "manualmente"

- Sirve para crear un "esqueleto" que facilite la creación de un módulo nuevo siguiendo la estructura de carpetas estándar.
- Recuerda que la carpeta del módulo debe estar en el addons_path

- Su uso sería así:
  1. Abre un terminal como administrador
  2. Accede a la carpeta de Odoo
  3. Ejecuta la siguiente orden:

```
"<ruta_Python>" "<ruta_odoo>" scaffold <nombre_modulo> <carpeta_modulo>
```

- Recuerda que la carpeta del módulo debe estar en el addons_path

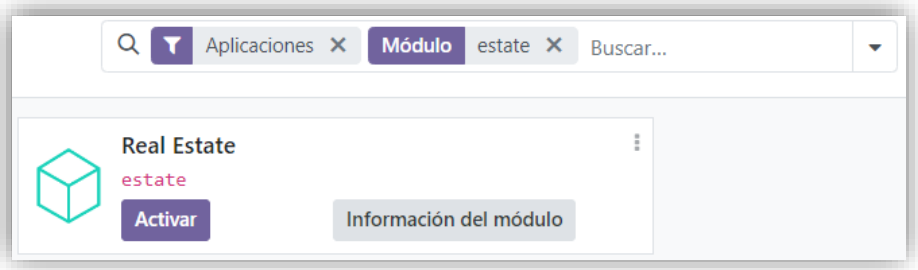
(p.e. elimina de 'data' y 'demo' los ficheros que no existan)

Capítulo 3 – Nueva Aplicación

- Reinicia el servidor
  - ➢Si estás trabajando con Docker:
    - a) Reiniciando el servicio desde Docker Desktop
    - b) Desde un terminal con la opción restart
  - ➢Si usas el instalador “oficial”:
    - a) Abriendo la aplicación ‘Servicios’ y seleccionando la opción ‘Iniciar’
    - b) Ejecutando Odoo desde VSC si lo tienes vinculado (Ejecutar > Ejecutar sin depuración)

el servicio ‘odoo-server’ debe estar parado

- Actualiza la lista de aplicaciones y busca ‘estate’



Capítulo 4 – Modelo

ORM (*Object Relational Mapping*): Crea vínculos entre los objetos en mi aplicación y las correspondientes tablas en la base de datos)

- **ORM** (doc): *Business object* = clase Python extendiendo 'Model'

  Ejemplo:

```
from odoo import models
class TestModel(models.Model):
    _name = "test.model"
```

crearía la tabla 'test_model' en la BD (al menos se debe definir el campo '_name')

- Crear el modelo para 'estate_property'
  - fichero 'estate/models/estate_property.py'

```
# -*- coding: utf-8 -*-
from odoo import fields, models
class EstateProperty(models.Model):
    _name = "estate.property"
    _description = "Real Estate Property"
```

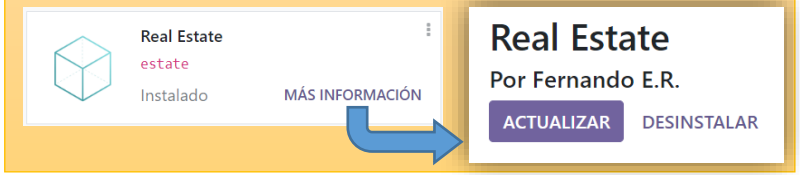
  - fichero 'estate/models/__init__.py':

```
from . import estate_property
```

  - fichero 'estate/__init__.py':

```
from . import models
```

Cualquier cambio requiere **reiniciar** el servidor y **actualizar** el módulo



Capítulo 4 – Campos (1/2)

- Información a guardar en el modelo (doc)
  Ejemplo: `name = fields.Char()`

- Tipos de campos:
  - ***simples***: Boolean, Float, Char, Integer, Text, Date, Selection,…
  - ***relacionales*** *(los veremos más adelante)*

- Añadir campos simples al modelo 'estate_property':
  - **name** (Char), **description** (Text), **postcode** (Char),**date_availability** (Date), **expected_price** (Float), **selling_price** (Float), **bedrooms** (Integer), **living_area** (Integer), **facades** (Integer), **garage** (Boolean), **garden** (Boolean), **garden_area** (Integer), **garden_orientation** (Selection: "North", "South", "East" and "West")

```
garden_orientation = fields.Selection(
    string='Garden Orientation',
    selection=[('N','North'),('S','South'),
               ('E','East'),('W','West')],
    help="Orientation of the property garden")
```

Lista de pares (*value*, *label*)

Capítulo 4 – Campos (2/2)

- Atributos como parámetros al crear el campo:
  - **string="…"** (etiqueta en UI, default: name)
  - **required=True** (not null, default: False)
  - **help="…"** (tooltip en UI, default: '')
  - **index=True** (DB index, default: False)
- Hay campos que se crean automáticamente (id, create_date,…) (doc)

- Configura los campos *name* y *expected_price* como *required*

Capítulo 5 – Seguridad

- Fichero de datos (CSV) en la carpeta ‘security’
  - Añadir a la lista ‘data’ del __manifest__.py

    Se cargan en orden (**ojo si hay referencias**)
  - Si no hay reglas ningún usuario tendrá acceso

- Añadir reglas al módulo ‘estate’
  - Crear fichero ‘ir.model.access.csv’ y declararlo
  - Agregar las reglas de acceso

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_estate_property,access_estate_property,model_estate_property,base.group_user,1,1,1,1
```

id (external id.), **name** (of the ir.model.access), **model_id:id** (model_<model_name>), **group_id:id** (group), perm_**read**,perm_**write**,perm_**create**,perm_**unlink** (permissions)

Capítulo 5 – Seguridad

Aquí vemos lo mínimo para nuestra aplicación, y damos permiso a todos los usuarios, pero en el desarrollo de módulos más complejos se tendrán en cuenta todas las situaciones posibles.

odoo > addons > account > security > ir.model.access.csv

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_account_cash_rounding_readonly,account.cash.rounding,model_account_cash_rounding,account.group_account_readonly,1,0,0,0
access_account_cash_rounding_uinvoice,account.cash.rounding,model_account_cash_rounding,account.group_account_invoice,1,1,1,1
access_res_partner_group_account_manager,res_partner group_account_manager,model_res_partner,account.group_account_manager,1,0,0,0

access_res_currency_account_manager,res.currency account manager,base.model_res_currency,group_account_manager,1,1,1,1
access_res_currency_rate_account_manager,res.currency.rate account manager,base.model_res_currency_rate,group_account_manager,1,1,1,1

access_account_invoice_report_readonly,account.invoice.report_user,model_account_invoice_report,account.group_account_readonly,1,0,0,0
access_account_invoice_report_billing,account.invoice.report_billing,model_account_invoice_report,account.group_account_invoice,1,0,0,0
access_account_invoice_report,account.invoice.report,model_account_invoice_report,account.group_account_manager,1,0,0,0

access_account_incoterms_all,account.incoterms all,model_account_incoterms,base.group_user,1,0,0,0
access_account_incoterms_manager,account.incoterms manager,model_account_incoterms,account.group_account_manager,1,1,1,1

access_account_fiscal_position_product_manager,account.fiscal.position account.manager,model_account_fiscal_position,account.group_account_
access_account_fiscal_position_tax_product_manager,account.fiscal.position.tax account.manager,model_account_fiscal_position_tax,account.gr
access_account_fiscal_position_account_product_manager,account.fiscal.position account.manager,model_account_fiscal_position_account,accoun
access_account_fiscal_position,account.fiscal.position all,model_account_fiscal_position,base.group_user,1,0,0,0
access_account_fiscal_position_tax,account.fiscal.position.tax all,model_account_fiscal_position_tax,base.group_user,1,0,0,0
access_account_fiscal_position_account,account.fiscal.position all,model_account_fiscal_position_account,base.group_user,1,0,0,0

access_product_product_account_user,product.product.account.user,product.model_product_product,group_account_readonly,1,0,0,0
access_product_product_account_manager,product.product.account.manager,product.model_product_product,account.group_account_manager,1,1,1,1

access_account_bank_statement_group_readonly,account.bank.statement.group.invoice,model_account_bank_statement,account.group_account_readon
access_account_bank_statement_group_invoice,account.bank.statement.group.invoice,model_account_bank_statement,account.group_account_invoice
access_account_bank_statement_line_group_readonly,account.bank.statement.line.group.invoice,model_account_bank_statement_line,account.group
access_account_bank_statement_line_group_invoice,account.bank.statement.line.group.invoice,model_account_bank_statement_line,account.group_
access_account_bank_statement,account.bank.statement,model_account_bank_statement,account.group_account_user,1,1,1,1
access_account_bank_statement_line,account.bank.statement.line,model_account_bank_statement_line,account.group_account_user,1,1,1,1
```

Capítulo 6 – UI actions (1/2)

- Ficheros de datos (XML) en carpeta 'views'
  - Definen Menus, Actions y Views
  - Declararlos en `__manifest__.py` (**recordad**: se cargan en orden)

- Actions (doc)

  esto es lo que vamos a ver ahora

  - Se activan haciendo clic en menús, botones o como acción contextual sobre un objeto
  - Lo podemos ver como un vínculo entre el *menu* y el *model*
  - *Ejemplo*: **ir.actions.act_window**

    La acción más común, usada para presentar visualizaciones de un *model* a través de *views*: una *window action* define un conjunto de tipos de vistas para un modelo.

Capítulo 6 – UI actions (2/2)

- Crea la acción para ‘estate.property’
  - Crea y declara estate/views/estate_property_views.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<odoo>
    <record id= "estate_property_action" model="ir.actions.act_window">
        <field name="name">Properties</field>
        <field name="res_model">estate.property</field>
        <field name="view_mode">tree,form</field>
    </record>
</odoo>
```

Identificador de la acción

Tipo de acción

Nombre de la acción

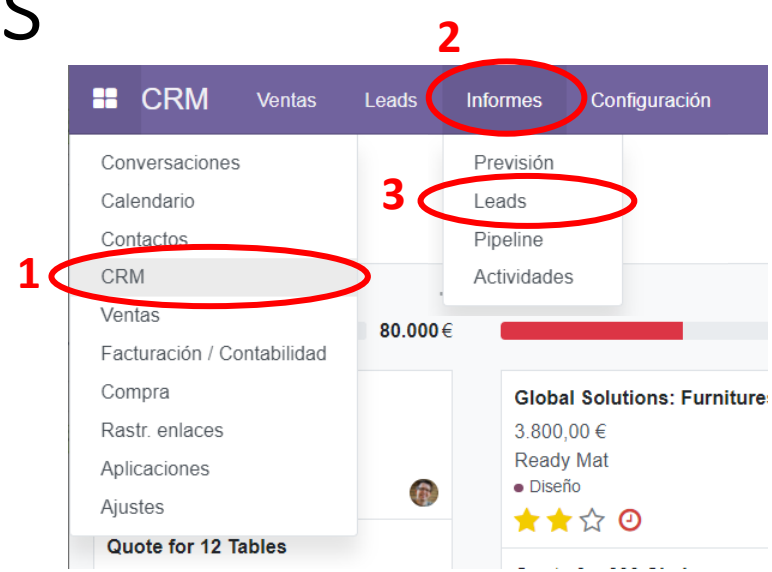
Modelo implicado

Vistas a usar con esta acción

Desde Odoo 18 ya no se usa la etiqueta “tree” para referirse a la vista de lista, tendríamos que cambiarlo por list.

Capítulo 6 – UI menus

- Hay 3 niveles de Menús (doc)
  1. Root menu
  2. First level menu
  3. Action menus



- Crea los menús para 'estate.property'
  - ➢Crea y declara estate/views/estate_menus.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<odoo>
    <menuitem id="estate_menu_root" name="Real Estate">
        <menuitem id="estate_advertisement_menu" name="Advertisements">
            <menuitem id="estate_property_menu_action" action="estate_property_action"/>
        </menuitem>
    </menuitem>
</odoo>
```

1 2 3

El 3er *name* se coge de la *action*

"To reduce the complexity in declaring a menu (ir.ui.menu) and connecting it to the corresponding action, we can use the <menuitem> shortcut"

Capítulo 6 – UI fields (1/2)

Puede que tengas que refrescar el navegador y limpiar la caché

- Siempre necesitamos un poco de ajuste fino para los campos, por ejemplo, para configurar:
  - valores *default*
  - solo-lectura
  - que no se copien al duplicar registro
  - …

- Ajusta los atributos del modelo (doc)
  - haz que…
    - ✓ 'selling_price' sea read-only y que no se copie al duplicar el registro
    - ✓ el valor por defecto de 'bedrooms' sea 2
    - ✓ 'date_availability' no se copie y por defecto sea 3 meses desde la creación

```
selling_price = fields.Float(
    "Selling Price",
    copy=False, readonly=True)
```

```
... ,default=lambda self: self._default_date_availability(), ...
```

```
from dateutil.relativedelta import relativedelta
…
def _default_date_availability(self):
    return fields.Date.context_today(self) + relativedelta(months=3)
```

Capítulo 6 – UI fields (2/2)

Puede que tengas que refrescar el navegador y limpiar la caché

- Campos reservados (doc)
  - Usa el campo '**active**' ajustando el valor por defecto a `True`

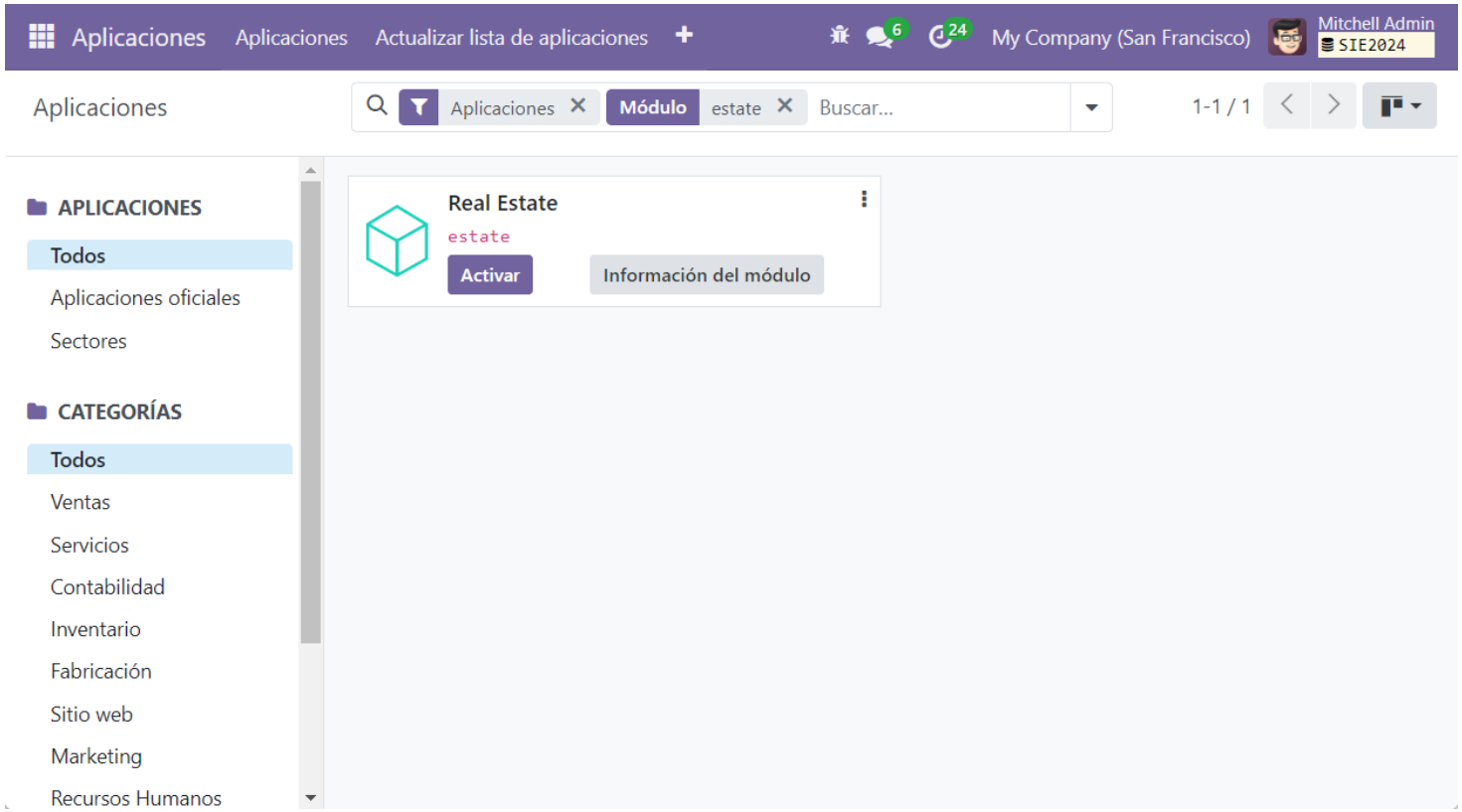
Si *active* es *False* el registro NO aparece en las listas y búsquedas a menos que busquemos específicamente los que tienen *active=False*

  - Usa el campo '**state**' con los siguientes ajustes:
    - ✓ Valores posibles: [New, Offer Received, Offer Accepted, Sold, Canceled]
    - ✓ default='New'
    - ✓ Requerido
    - ✓ Que no se copie al duplicar registros

selection

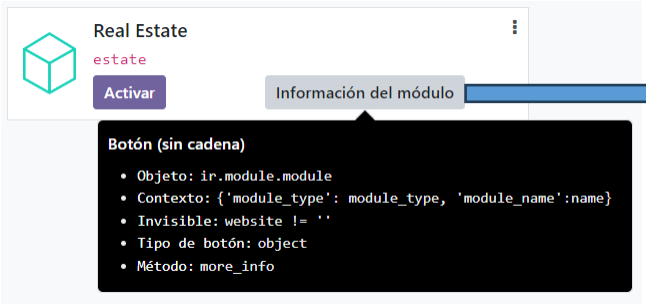
Recopilemos…

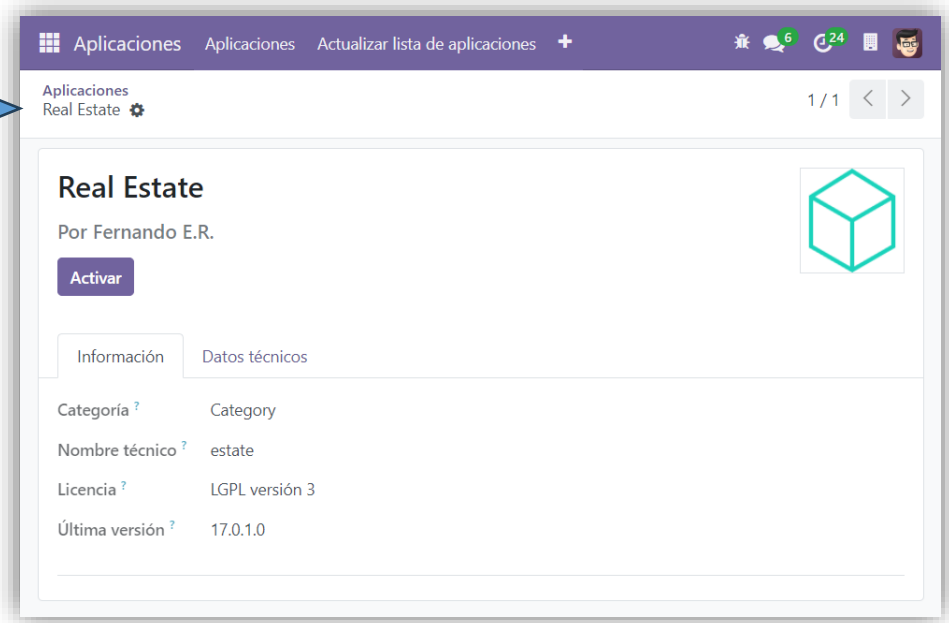
- ¿Qué hemos conseguido hasta ahora?



Recopilemos…

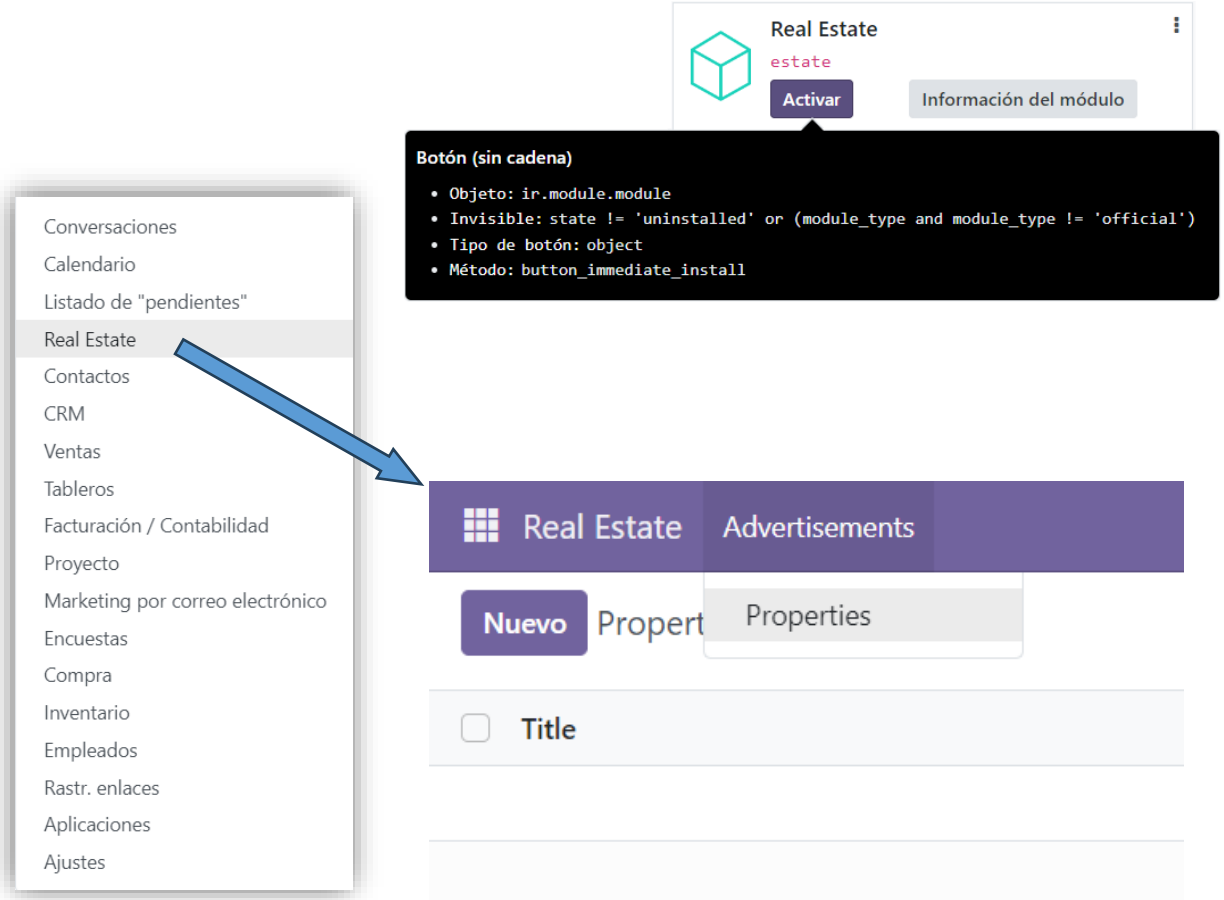
- ¿Qué hemos conseguido hasta ahora?





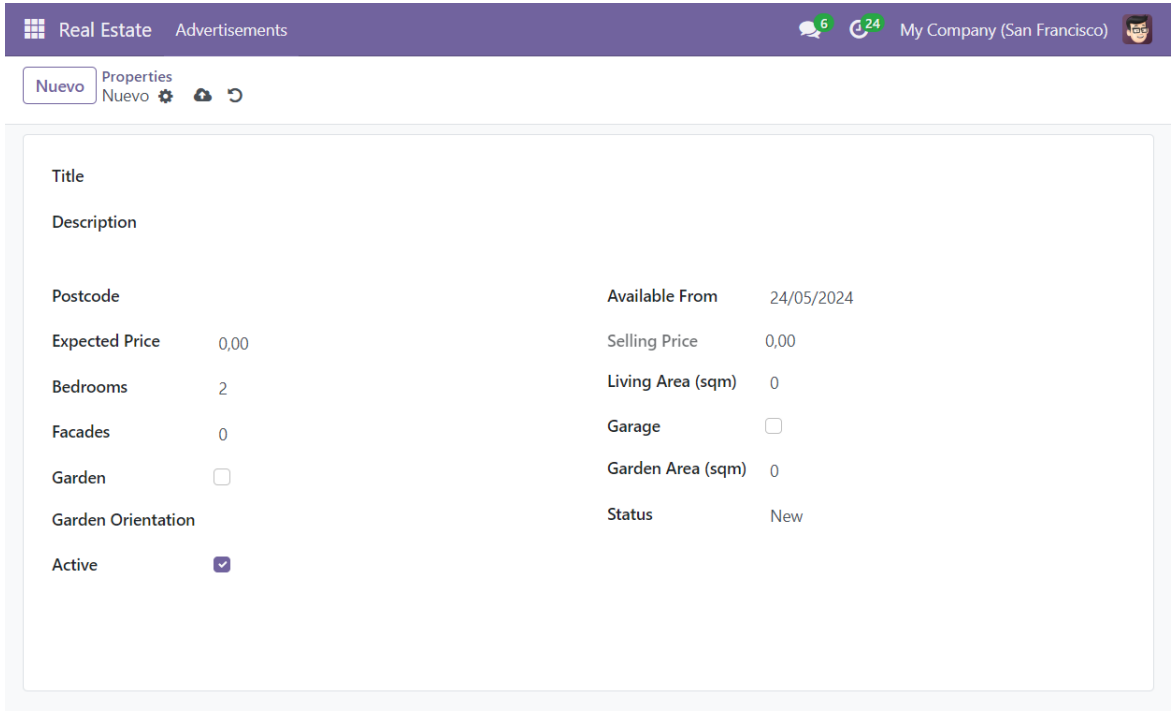
Recopilemos…

- ¿Qué hemos conseguido hasta ahora?



Recopilemos…

- ¿Qué hemos conseguido hasta ahora?



Capítulo 7 – Views básicas

son registros del modelo **ir.ui.view**

- No nos conformemos con las *views* por defecto
  - ➢No serán suficientes en ningún escenario “real”
- Definidas en ficheros XML en la carpeta ‘views’
  - ➢Un fichero por cada *model* (mezclan *actions* y *views*)
  - ➢A continuación, crearemos *views* para ‘estate_property’ ampliando ‘estate_property_views.xml’ con nuevos *records*

Mismo fichero de ‘`estate_property_action`’

- Queremos:
  - ✓En la lista, mostrar algo más que el nombre.
  - ✓En el form, agrupar los campos.
  - ✓En la búsqueda, poder buscar por más campos, filtrar las propiedades “disponibles” y agrupar por ‘postcode’.

Capítulo 7 – List

- *List view*: Muestra los datos en una tabla (doc1;doc2)

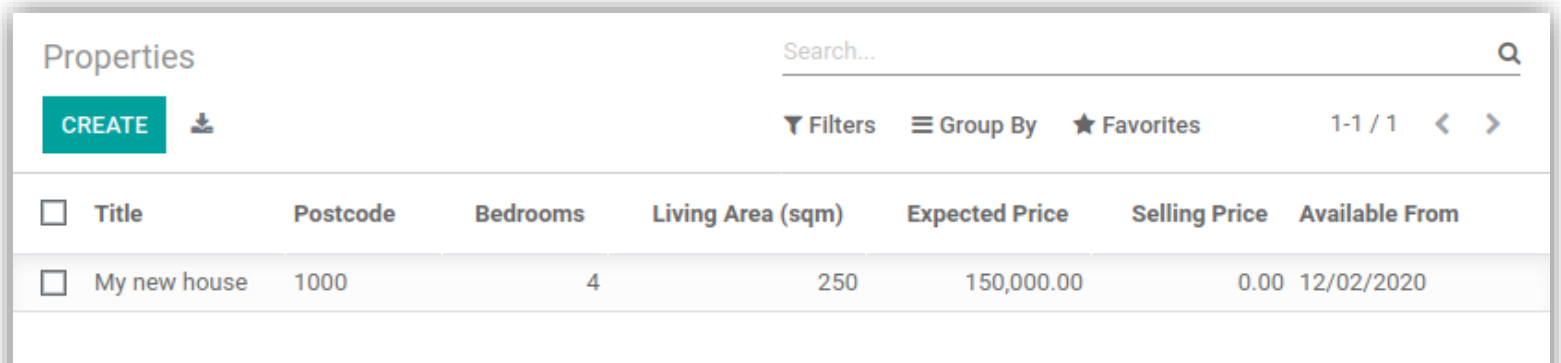
asegúrate de que los id de las vistas son únicos

```
<record id="estate_property_view_tree" model="ir.ui.view">
    <field name="name">estate.property.tree</field>
    <field name="model">estate.property</field>
    <field name="arch" type="xml">
        <tree string="Properties">
            <field name="name"/>
            ...
```

list desde Odoo 18

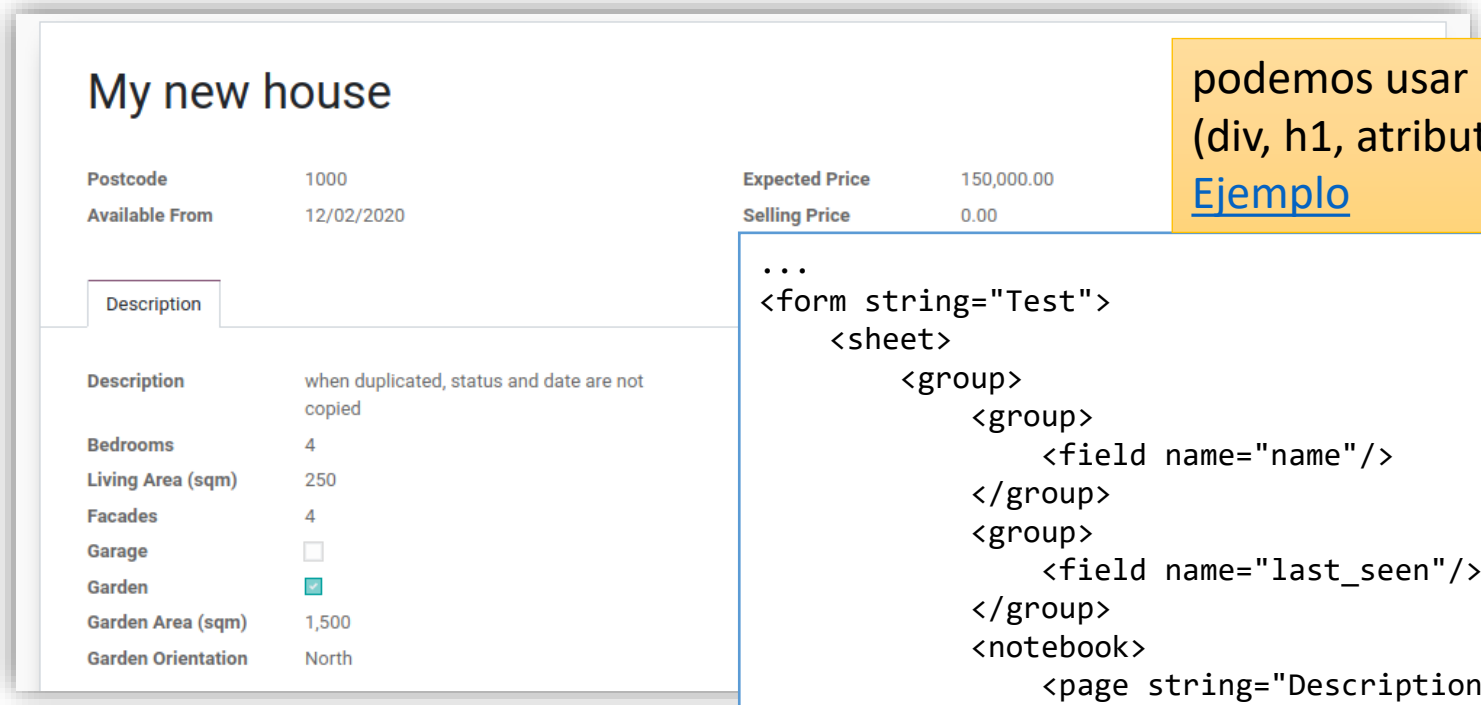
El campo 'arch' es el que indica el tipo de vista que se está definiendo:

<field name="arch" type="xml">
*<**form** ...>, <**list** ...>, <**graph** ...>, ...*
</field>



Capítulo 7 – Form

- *Form view*: Para crear/editar un único registro (doc)
- Compuestos por:
  - ➢elementos estructurales de alto nivel (*groups* & *notebooks*)
  - ➢elementos interactivos (*buttons* & *fields*)

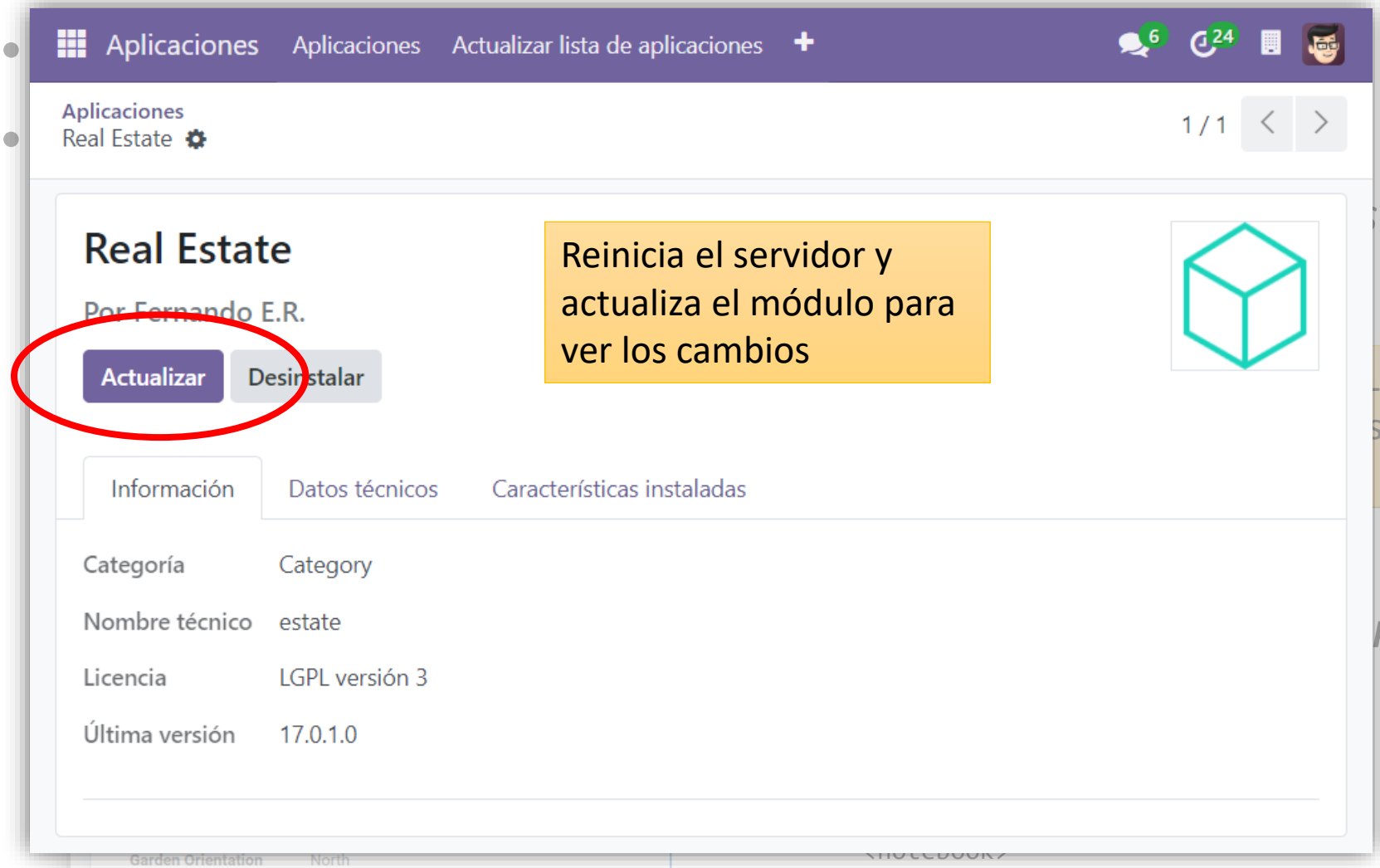


podemos usar HTML (div, h1, atributo class,...) Ejemplo

Ejemplo

```
...
<form string="Test">
    <sheet>
        <group>
            <group>
                <field name="name"/>
            </group>
            <group>
                <field name="last_seen"/>
            </group>
            <notebook>
                <page string="Description">
                    <field name="description"/>
```

Capítulo 7 – Form

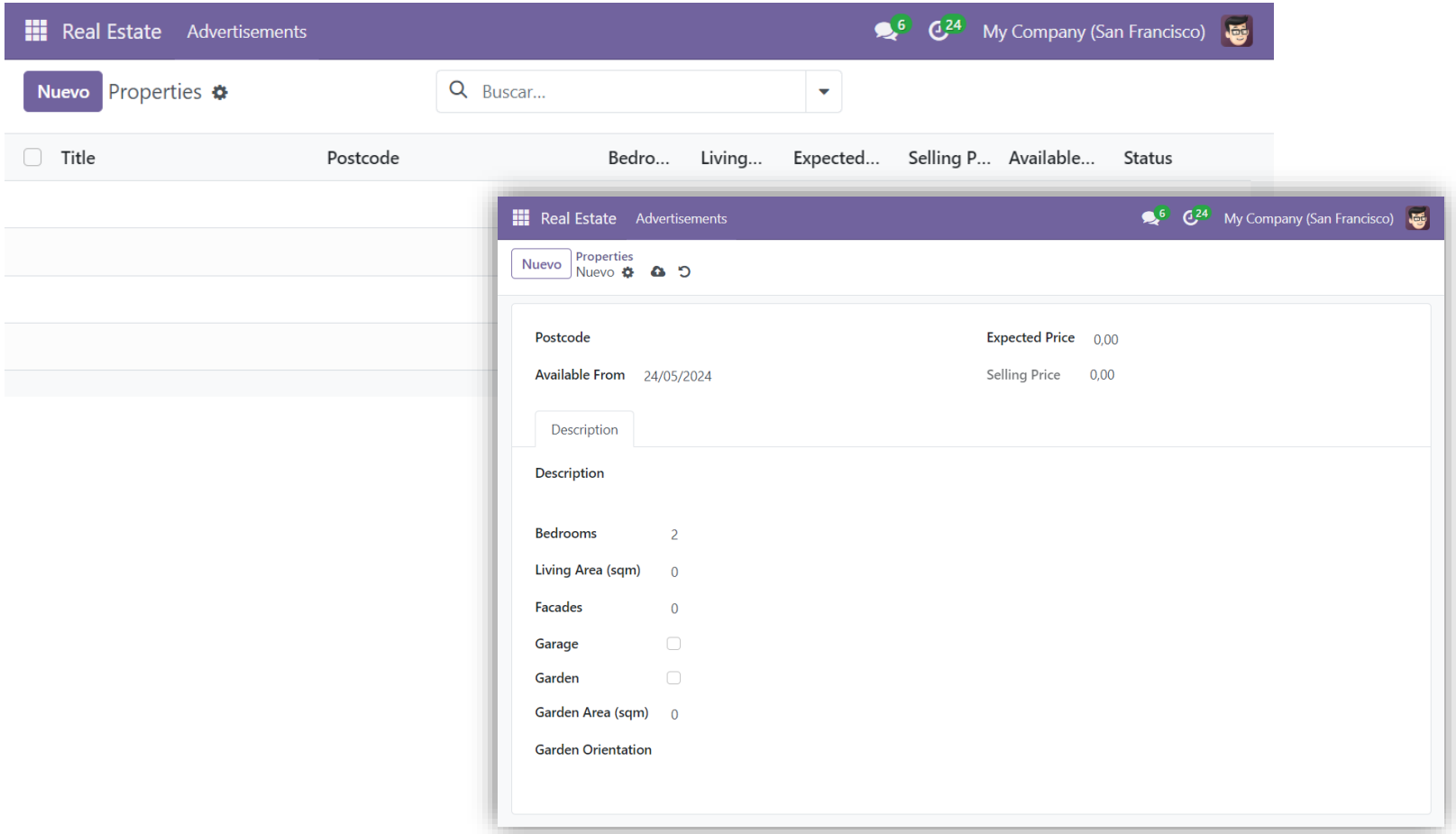


Reinicia el servidor y actualiza el módulo para ver los cambios

```
<page string="Description">
    <field name="description"/>
```

Recopilemos…

- ¿Qué hemos conseguido hasta ahora?

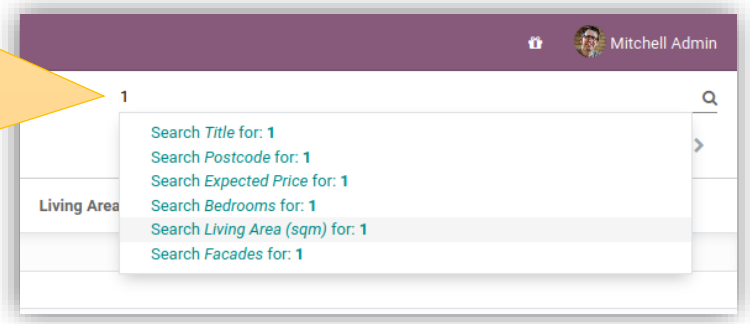


Capítulo 7 – Search (1/3)

- *Search view*: para filtrar el contenido de otras vistas (doc)
  - En su versión más simple solo indicamos los campos para los que queremos que aparezca un **atajo** cuando escribimos algo

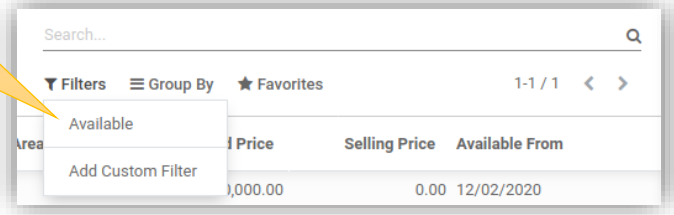
Escribimos algo en el cuadro de búsqueda y aparecen atajos para filtrar por campo1, campo2,… usando ese valor:

```
<search string="Tests">
    <field name="campo1"/>
    <field name="campo2"/>
    ...
</search>
```

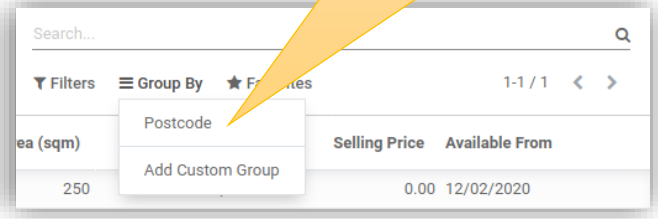


  - También definimos **filtros** y **agrupaciones**

Filtro para ver solo las propiedades disponibles



Opción para agrupar por código postal



Capítulo 7 – Search (2/3)

- **Dominios**: Condición para seleccionar un subconjunto de registros (doc)
  - Tripleta (campo, operador, valor)
  - Podemos combinar varias tripletas:
    - ✓ Se usa una lista de tripletas: [(c1,op1,v1), (c2,op2,v2),…, (c2,op2,v2)]
    - ✓ Por defecto (si no indicamos operador) se combinan con AND ('&')
    - ✓ Operadores disponibles: '&', '|', '!'
    - ✓ Para la combinación de tripletas se usa la notación prefija

¡OJO! En las tripletas se usa notación infija

- *Ejemplo*:
  - servicios o productos con precio fuera del rango [1000..2000)

"['|', ('product_type', '=', 'service'), '!', '&', ('unit_price', '>=', 1000), ('unit_price', '<', 2000)]"

Capítulo 7 – Search (3/3)

- Añade:
  - **atajos** para los campos “name”, “postcode”, “expected_price”, “bedrooms”, “living_area” y “facades”
  - **filtro** para propiedades que estén disponibles para la venta (‘state’ = “New” o “Offer Received”)
  - poder **agrupar** por ‘postcode’ (con <filter> pero en lugar de ‘domain’ usar atributo ‘context’)

Atajo

Filtro

Agrupación

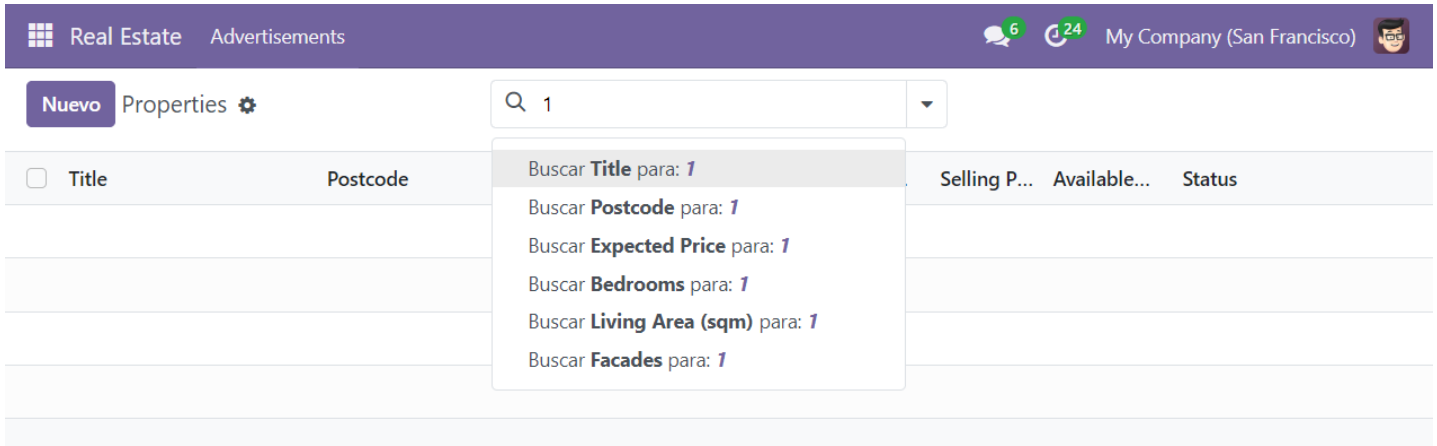
```
<search …>
  <field name=… />
  <separator/>
  <filter string=…  name=…  domain=…/>
  <group expand="1"  string="Group By">
    <filter  string=…  name=…  context="{'group_by’:…}"/>
  </group>
…
```

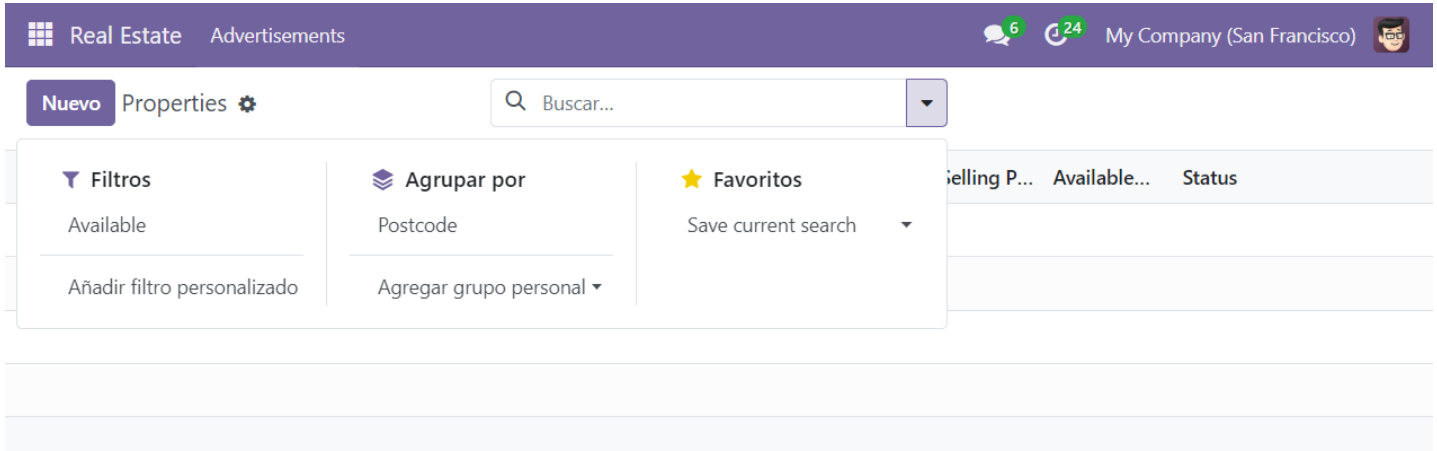
Ver: ejemplo

- Añadir uno a uno los campos y etiquetas para comprenderlas
- Usar odoo-bin con el parámetro --dev xml para solo tener que refrescar el navegador

Recopilemos…

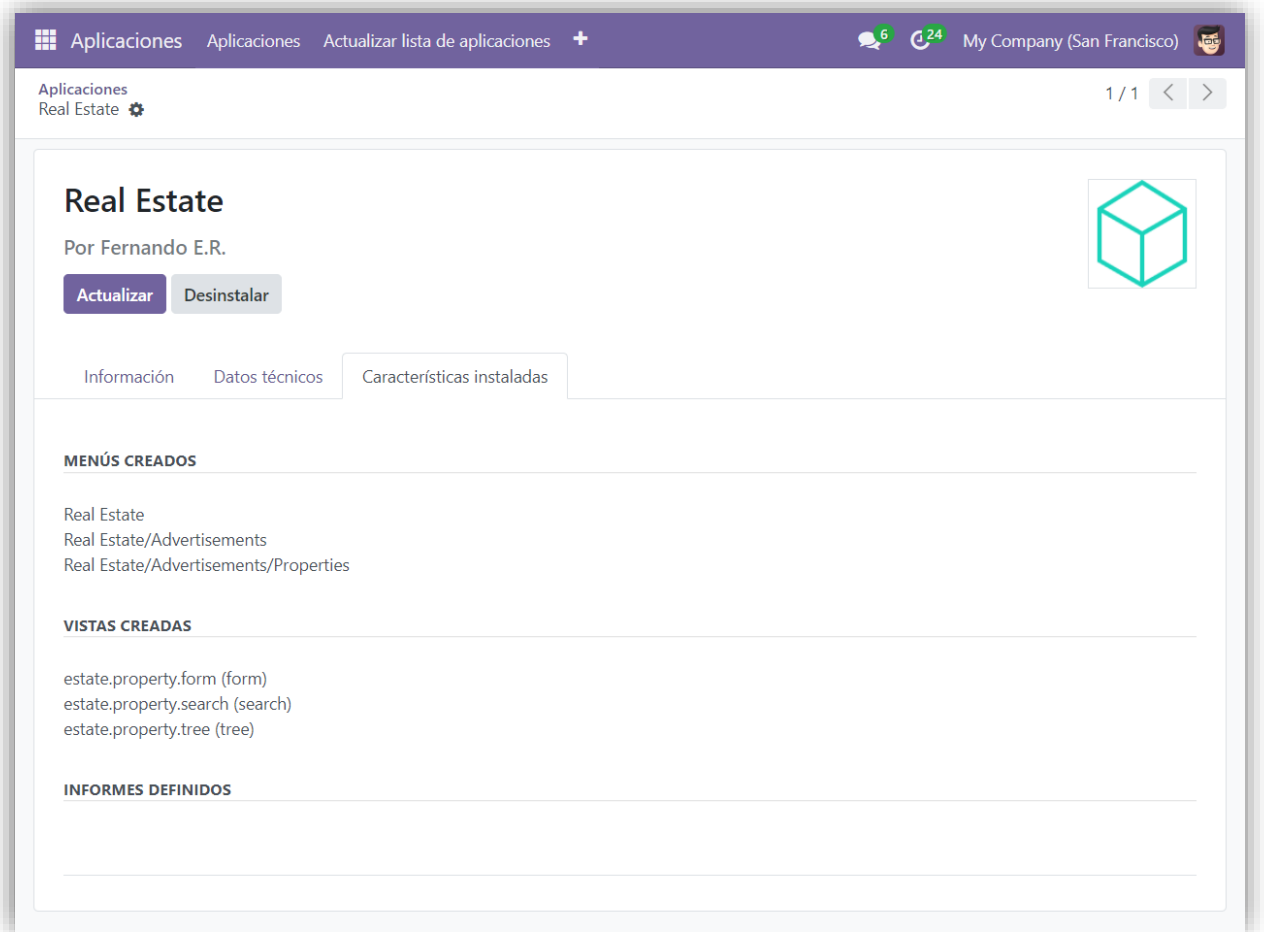
- ¿Qué hemos conseguido hasta ahora?





Recopilemos…

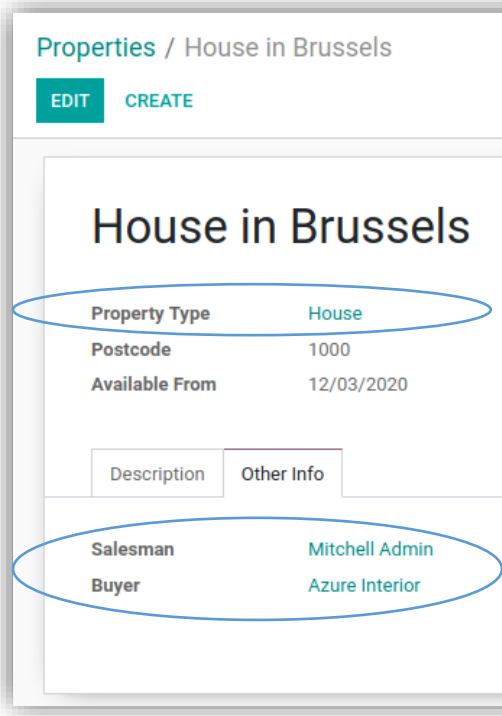
- ¿Qué hemos conseguido hasta ahora?



Repaso

1. ¿Cuáles son las características básicas de un módulo en Odoo?
2. ¿Cómo preparamos el entorno para desarrollar un módulo de Odoo?
3. ¿Qué es la opción scaffold de odoo-bin?
4. ¿Qué es lo mínimo que tenemos que hacer para definir un módulo nuevo en Odoo que pueda aparecer en la lista de aplicaciones de Odoo?
5. ¿Podemos iniciar Odoo desde Visual Studio Code?
6. ¿Cómo se define un modelo para un módulo de Odoo?
7. ¿Cómo se definen los atributos de un modelo en Odoo?
8. ¿Cómo se definen los permisos de acceso al crear un módulo en Odoo?
9. ¿Cómo se definen los menús al crear un módulo en Odoo?
10. ¿Cuáles son algunas de las características que se le pueden asignar a los atributos de un modelo en Odoo?
11. ¿Cuáles son campos reservados de un modelo en Odoo y cómo podemos configurarlos?
12. ¿Cómo se configuran las vistas de tipo lista y formulario de un módulo en Odoo?
13. ¿Cómo se añaden atajos, filtros y agrupaciones para facilitar la búsqueda de registros al crear un módulo en Odoo?

Capítulo 8 – Many2one



- Sobre cada propiedad, queremos:
  - ➢el tipo: *house, apartment, penthouse, castle*…
  - ➢el cliente que compró
  - ➢el agente que la vendió
- Relación Many2one (doc)
  - ➢El nombre del campo termina en **`_id`**
- Crea un nuevo modelo 'estate.property.type'
  - ✓con el campo: 'name' (Char, required)
  - ✓Añade *action*, *menu* y *view*

No olvidar los __init__.py, __manifest__.py , permisos,…

- Actualiza el modelo 'estate.property' añadiendo:
  - ✓Un campo 'property_type_id' (añadir también a las *views*)
  - ✓Campos 'comprador' y 'vendedor' en una pestaña nueva del *form*
    - 'vendedor' por defecto = usuario actual (ejemplo)
    - 'comprador' no debe copiarse

Capítulo 8 – Many2one

- Sobre cada propiedad, queremos:
  - R
    - El nombre del campo termina en **_id**
  - Cr
  - Ac
    - 'comprador' no debe copiarse

In Odoo, there are two models which we commonly refer to:

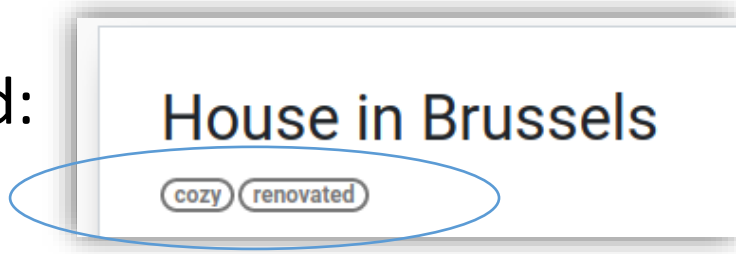
- `res.partner`: a partner is a physical or legal entity. It can be a company, an individual or even a contact address.
- `res.users`: the users of the system. Users can be "internal", i.e. they have access to the Odoo backend. Or they can be "portal", i.e. they cannot access the backend, only the frontend (e.g. to access their previous orders in eCommerce).

Nota

The object `self.env` gives access to request parameters and other useful things:

- `self.env.cr` or `self._cr` is the database *cursor* object; it is used for querying the database
- `self.env.uid` or `self._uid` is the current user's database id
- `self.env.user` is the current user's record
- `self.env.context` or `self._context` is the context dictionary
- `self.env.ref(xml_id)` returns the record corresponding to an XML id
- `self.env[model_name]` returns an instance of the given model

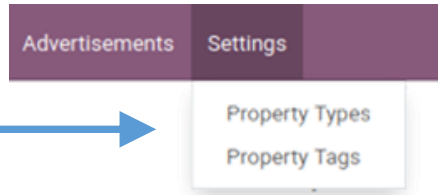
Capítulo 8 – Many2many



- Ahora queremos para una propiedad:
  - ➢ poder asociarle etiquetas
  - ➢ la lista de ofertas recibidas
- Sobre la relación Many2many (doc)
  - ➢ Relación bidireccional múltiple
  - ➢ El nombre del campo termina en **`_ids`**
  - ➢ El acceso debe hacerse en un bucle
  - ➢ Será un *recordset*: Lista de registros

Permite usar operaciones de listas Python como `len()`, `iter()`, y otras (ej.: rs1 | rs2)

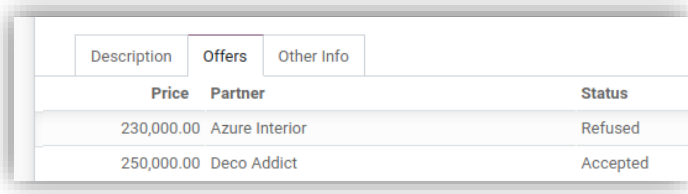
- Crea un nuevo modelo 'estate.property.tag'
  - ✓ con el campo: 'name' (Char, required)
  - ✓ los menús



- Actualiza el modelo 'estate.property'
  - ✓ campo 'tag_ids' (también las *views*: *form* y *tree*)

Usar widget "many2many_tags" (ver ejemplo)

Capítulo 8 – One2many



- Ahora queremos para una propiedad:
  - ➢ Poder asociarle ofertas de compra
- Sobre la relación One2many (doc)
  - ➢ Relación **virtual**, **inversa** de una *Many2One* **ya existente**

```
test_ids = fields.One2many("test.model", "partner_id", string="Tests")
```

  - ➢ ***Comodel***: modelo donde está definido la relación *Many2one*
  - ➢ El nombre del campo termina en **`_ids`**
  - ➢ El acceso se hace en un bucle (sobre recordset)
- Crea un nuevo modelo 'estate.property.offer'
  - ✓ *Campos*: price (Float), status (Selection: [Accepted, Refused], no copiar), partner_id (Many2one (res.partner), required), property_id (Many2one (estate.property), required)
  - ✓ También las *views*: *form* y *tree*, con price, partner_id y status
- Actualiza el modelo 'estate.property'
  - ✓ Campo 'offer_ids' (y también *view form*)

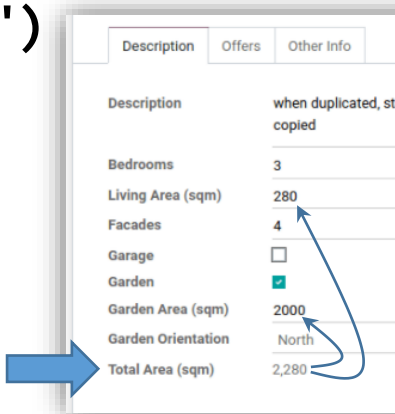
No requiere actions y menus, accedemos desde la propiedad

El valor se asigna automáticamente

Capítulo 9 – Computed fields (1/3)

- Relaciones entre campos del modelo
  - ➢ dependencia entre sus valores o para ayudar a rellenar forms
- Queremos para una propiedad:
  - ➢ El área total (living_area + garden_area)
- Campos computados (doc)(ejemplo)
  - ➢ el valor no proviene de la BD sino que se calcula al vuelo
  - ➢ asignar atributo 'compute' a método privado (prefijo '_')
  - ➢ el método debe ejecutar los cálculos con **`for record in self:`**
  - ➢ indicar las dependencias con **`@api.depends("field")`**
    - ❖ *field* será el campo del que dependemos (o campos)
- Crea el campo 'total_area' en 'estate.property'
  - ✓ se calcula como living_area + garden_area
  - ✓ añádelo al *form view* en 'Descripción'

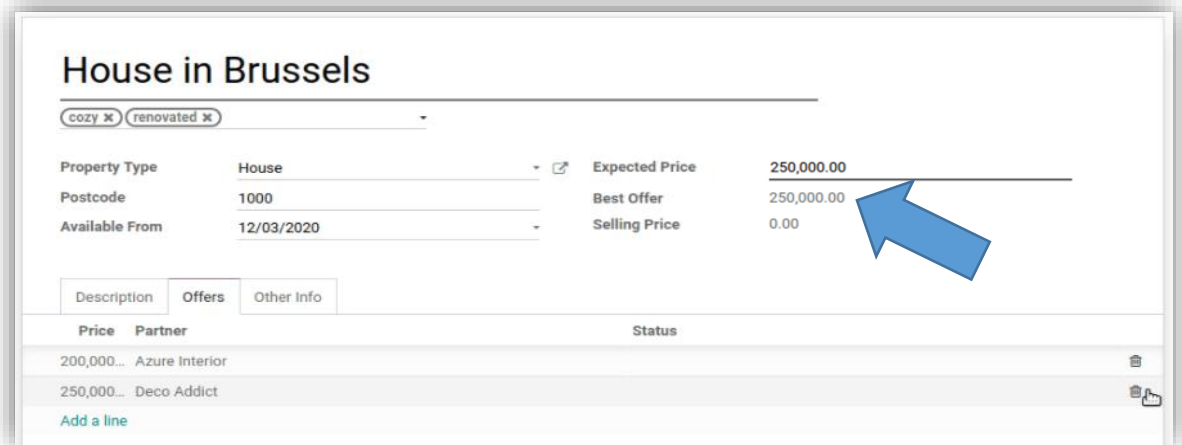
`self` es una colección (recordset)



Capítulo 9 – Computed fields (2/3)

- También válido para campos de tipo relación (ejemplo)
  - ➢Pueden ser *Many2one*, *Many2many* o *One2many*
- Queremos para una propiedad:
  - ➢la mejor oferta
- Crea el campo 'best_price' en estate.property
  - ✓se calcula como el máximo precio de las ofertas
  - ✓añádelo al *form view*

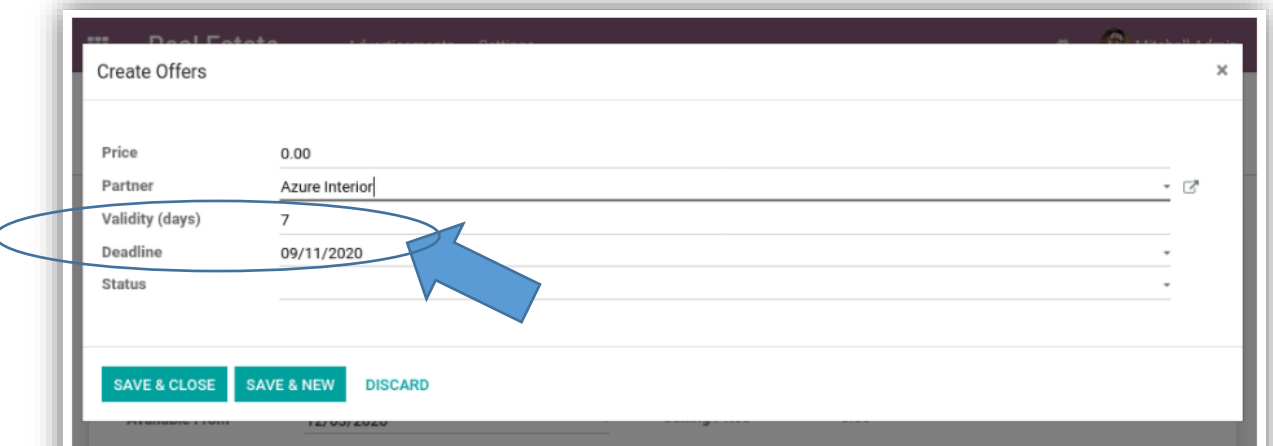
ver uso de `mapped()` (ejemplo)



Capítulo 9 – Computed fields (3/3)

- Por defecto son read-only pero existe el atributo 'inverse = "*_función*" ' (ejemplo)
  - ➢ Al fijar un valor nosotros actualiza las dependencias
- Queremos para una oferta:
  - ➢ La validez (días) ↔ fecha límite
- Actualiza el modelo 'estate.property.offer':
  - ✓ Añade los campos: validity (Integer, default=7), date_deadline (Date)
  - ✓ date_deadline se calcula como create_date + validity
  - ✓ Define la función inversa para que el usuario pueda fijar ambos campos
  - ✓ Añade los campos a form y tree

- pueden depender de otros c.f.
- ¿usarlos en búsquedas?
  - atributo 'search'
  - atributo store=True
- **¡OJO al rendimiento!**

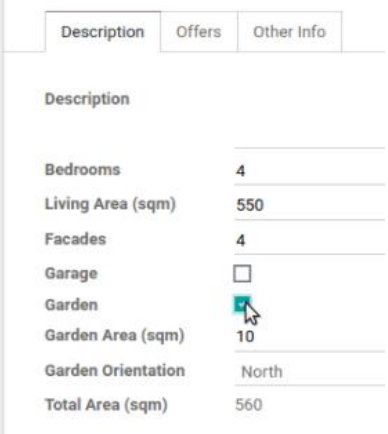


- *create_date* se rellena al crearse el registro ➔ añadir código que evite errores durante la creación (if …)
- El método *inverse* se llama al guardar, el *compute* con cada cambio de las dependencias

Capítulo 9 – Onchanges

- Al rellenar formularios queremos que un campo se actualice al cambiar otro, sin grabar en la BD (doc)
- Definimos un método donde:
  - *self* representa el registro en el form view
  - se decora con **`@api.onchange("field")`**
    - *field* indica el campo por el que se dispara
- Actualiza 'estate.property' con un *onchange*:
  - si 'garden' es True: 'garden area'=10 y 'orientation'=North
  - si no: borrar 'garden area' y 'orientation'
- Se pueden mostrar mensajes (ejemplo)
- No crear lógica de negocio (solo UI)
  - Al crear registros programando no se aplican

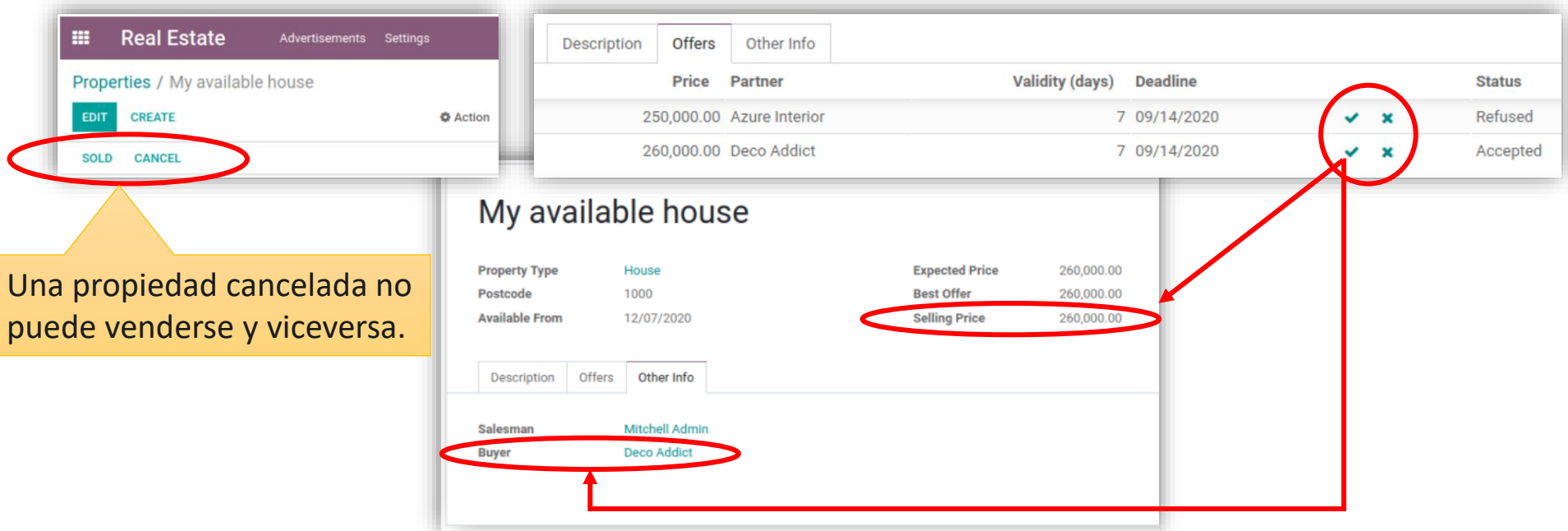
Los c.f. funcionan en el servidor, y aquí hablamos de la capa de presentación



Capítulo 10 – Action buttons #1

- Queremos botones (doc) para:
  - ➢Cancelar o marcar una propiedad como “vendida”
  - ➢Aceptar o rechazar ofertas
    (Si se acepta fijaremos el precio de venta y el comprador)

También podríamos andar modificando los campos, pero mejor automatizarlo



Una propiedad cancelada no puede venderse y viceversa.

Capítulo 10 – Action buttons #2

- Actualiza el modelo ‘estate_property’

método publico (sin prefijo _) ➔ accesible desde GUI

```
def action_sold(self):
    if "canceled" in self.mapped("state"):
        raise UserError("Canceled properties cannot be sold.")
    return self.write({"state": "sold"})
def action_cancel(self):
    if "sold" in self.mapped("state"):
        raise UserError("Sold properties cannot be canceled.")
    return self.write({"state": "canceled"})
```

ver función ‘UserError’ (doc) y excepciones (doc):
`from odoo.exceptions import UserError`

- Actualiza ‘estate_property_view_form’

```
...
<form>
  <header>
    <button name="action_sold" type="object" string="Sold"/>
    <button name="action_cancel" type="object" string="Cancel"/>
  <header>
  ...
```

Import necesario: `from odoo.exceptions import ValidationError`

Capítulo 10 – Action buttons #3

- Actualiza el modelo ‘estate_property_offer’

```
def action_accept(self):
    if "accepted" in self.mapped("property_id.offer_ids.state"):
        raise UserError("An offer has already been accepted.")
    self.write({"state": "accepted"})
    return self.mapped("property_id").write({
        "state": "offer_accepted",
        "selling_price": self.price,
        "buyer_id": self.partner_id.id})
def action_refuse(self):
    return self.write({"state": "refused"})
```

- Actualiza ‘estate_property_view_form’

```
...
<tree string="Property Offers">
  ...
  <button name="action_accept" type="object" title="Accept" icon="fa-check"/>
  <button name="action_refuse" type="object" title="Refuse" icon="fa-times"/>
  ...
```

consultar iconos disponibles en: https://fontawesome.com/v4/icons/

También se puede ligar un *button* con una *action*. Ejemplo:
`<button type="action" name="%(test.test_model_action)d" string="My Action"/>`

Capítulo 11 – Constraints (SQL)

- Evitar datos erróneos (p.e. un precio negativo):
  - SQL constraints (doc)
  - Python constraints (doc)

Ver también PostgreSQL (doc) p.e. para consultar posibles definiciones de constraints SQL

Poner una restricción y actualizar el módulo cuando hay datos que la incumplen provocará un error

- SQL
  - Consiste en el campo **`_sql_constraints=[`*`cons1,cons2,…`*`]`** siendo cada *cons* una tupla: `(name, definición, mensaje)`
  - Queremos precios positivos y nombres de types/tags únicos
    - Añadir el campo **`_sql_constraints`** a los modelos con:

'estate_property'

```
("check_expected_price", "CHECK(expected_price > 0)", "..."),
("check_selling_price", "CHECK(selling_price >= 0)", "...")
```

'estate_property_offer'

```
("check_price", "CHECK(price > 0)", "...")
```

'estate_property_type'

```
("check_name", "UNIQUE(name)", "...")
```

'estate_property_tag'

```
("check_name", "UNIQUE(name)", "...")
```

Capítulo 11 – Constraints (Python)

Si podemos elegir entre constraints SQL y Python, es preferible SQL por eficiencia.

- Python
  - Para casos de restricciones más "complejas" (ejemplo)
  - Métodos decorados con `@api.constrains('field')` donde *`field`* es el campo que está involucrado
  - Queremos: *selling_price* >= 90% del *expected_price*
    - Actualiza el modelo 'estate_property':

Se verificará cada vez que cambie uno de estos campos

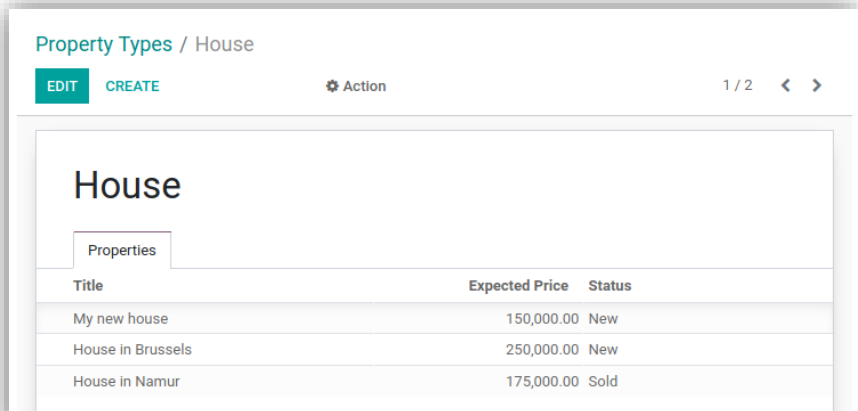
```
@api.constrains("expected_price", "selling_price")
def _check_price_difference(self):
    for p in self:
        if (not float_is_zero(p.selling_price, precision_rounding=0.01)
            and float_compare(p.selling_price, p.expected_price * 90.0 / 100.0,
                    precision_rounding=0.01) < 0):
                raise ValidationError("... mensaje ...")
```

Import necesario: `from odoo.exceptions import ValidationError`

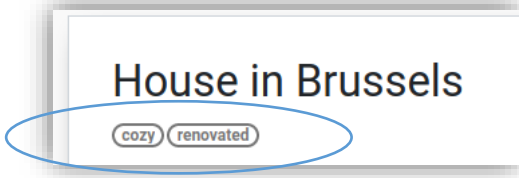
Capítulo 12 – Sprinkles (1/15)

- *Inline views*
  - ➢ Con una relación One2Many-Many2One como la de Property↔Offers usamos la *view* de Offer en el Form de Property, pero a veces queremos una *inline view* por ejemplo para mostrar solo algunos campos.
- Queremos para cada estate.property.type
  - ➢ listar las propiedades de ese tipo
- Actualiza el modelo estate.property.type
  - ✓ Añade el campo property_ids (One2many)
  - ✓ Añade una *inline view* a su *form view*:

```
<notebook>
    <page string="Properties">
        <field name="property_ids" readonly="1">
            <tree>
                <field name="name"/>
                <field name="expected_price"/>
                <field name="state"/>
            </tree></field></page></notebook>
```



Capítulo 12 – Sprinkles (2/15)

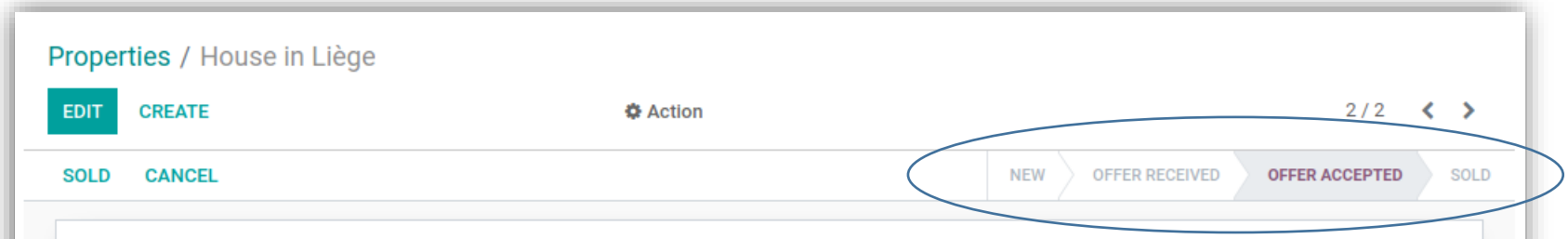


- *Field widgets* (doc)
  - ➢ Odoo selecciona el widget apropiado para cada campo, pero podemos especificar otro, por ejemplo antes con las etiquetas usamos “many2many_tags” para no mostrarlas como una lista.
- Queremos para cada estate.property
  - ➢ Mostrar el estado con “statusbar” (ejemplo)

¡OJO! NO incluir un campo más de 1 vez

- Actualiza la *form view* del modelo estate.property
  - ✓ Añade el widget ‘statusbar’ al campo ‘state’:

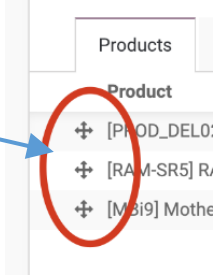
```
<field name="state" widget="statusbar"
    statusbar_visible="new,offer_received,offer_accepted,sold"/>
```



Capítulo 12 – Sprinkles (3/15)

- *List order* (doc)
  - ➢ Podemos decir en qué orden mostrar los registros listados de 3 formas:
    - a) En el ***model***: atributo `_order="id desc"` (ya siempre en este orden)
    - b) En la ***view***: atributo `default_order` (ejemplo)
    - **c) Manualmente**: campo Integer 'sequence' + *widget handle*
      1. Crear campo de tipo Integer 'sequence' (valor por defecto 1)
      2. Configurar _order="sequence,id,…" (*sequence* siempre el 1º)
      3. En la *view* usamos <field name="sequence" widget="handle"/>

Se convierte en **order by** (SQL)



- Queremos ordenar distintos listados
  - ➢ Aplicar un orden por modelo y/o de forma manual
- Añade a los siguientes modelos el atributo `_order`
  - ✓ estate.property → ID desc
  - ✓ estate.property.offer → Price desc
  - ✓ estate.property.tag → Name
  - ✓ estate.property.type → Name
- Actualiza el modelo estate.property.type
  - ✓ Sigue los pasos comentados antes para que se ordene también manualmente

Capítulo 12 – Sprinkles (4/15)

- *Attributes and options - Form*
  - El atributo ***options*** permite crear etiquetas de colores, evitar la creación/edición de registros relacionados, etc.
- Queremos que desde el *form view* de estate.property
  - Podamos asignar colores a las etiquetas
  - No pueda crearse o editarse estate.property.type
- Actualiza el módulo estate.property.tag
  - ✓ Añade el campo: `color = fields.Integer("Color Index")`
- Actualiza estate.property
  - ✓ Añade al *form view* los atributos *options* adecuados (doc)

```
<field name="tag_ids" widget="many2many_tags" options="{'color_field': 'color'}"/>
...
<field name="property_type_id" options="{'no_create': True, 'no_edit': True}"/>
```

Capítulo 12 – Sprinkles (5/15)

- *Attributes and options - Form*
  - ➢ Usar el campo *state* junto al atributo ***states*** permite ocultar botones condicionalmente en la *view*
- Queremos que en el *form view* de estate.property
  - ➢ Los botones aparezcan solo cuando tengan sentido
- Actualiza estate.property
  - ✓ Añade al *form view* los atributos *states* adecuados

```
<button name="action_sold" ... states="offer_accepted" ...
<button name="action_cancel" ... states="new,offer_received,offer_accepted" ...
```

Capítulo 12 – Sprinkles (6/15)

- *Attributes and options - Form*
  - El atributo ***attrs*** permite convertir *fields*, *buttons*, *groups*,… en *invisible*, *readonly* o *required* basándose en otros campos
  - *attrs* es un diccionario {propiedad:dominio}

¡OJO! Usar en *attrs* solo *fields* presentes en la *view* (podemos ocultarlos con el atributo `invisible="1"`)

- Queremos: a) para el *form view* de estate.property
  - Ocultar el área y la orientación del jardín cuando no hay jardín

```
<field name="garden_area"
    attrs="{'invisible': [('garden', '=', False)]}"/>
```

  - No permitir añadir ofertas a una propiedad con estado “Offer Accepted”, “Sold” o “Canceled”

```
<field name="offer_ids"
    attrs="{'readonly': [('state', 'in', ('offer_accepted’,...))]}"/>
```

Capítulo 12 – Sprinkles (7/15)

- *Attributes and options - Form*
  - El atributo ***attrs*** permite convertir *fields*, *buttons*, *groups*,… en *invisible*, *readonly* o *required* basándose en otros campos
  - *attrs* es un diccionario {propiedad:dominio}

¡OJO! Usar en *attrs* solo *fields* presentes en la *view* (podemos ocultarlos con el atributo `invisible="1"`)

- Queremos: b) para el *form view* de estate.property.offer
  - Ocultar los botones "Accept","Refuse" al fijar el estado de la oferta

```
<button name="action_accept" ...
    attrs="{'invisible': [('state', '!=', False)]}"/>
```

- Actualiza estate.property y estate.property.offer
  - ✓ Añade al *form view* los atributos *attrs* adecuados

Capítulo 12 – Sprinkles (8/15)

- *Attributes and options - List*
  - El atributo ***editable*** permite editar una lista sin abrir el Form view
  - El atributo ***optional*** permite ocultar el campo
- Queremos para estate.property.offer y estate.property.tag
  - Hacer las listas editables

```
<tree string="Property Offers" editable="bottom"
```

Añade debajo ('top' → arriba)

- Queremos para las listas de estate.property
  - Ocultar por defecto el campo `date_availability`

```
<field name="date_availability" optional="hide"/>
```

- Actualiza los modelos *offer*, *tag* y *property*
  - ✓ Añade a las *tree views* los atributos adecuados

Capítulo 12 – Sprinkles (9/15)

decoration-bf
decoration-it
decoration-danger
decoration-info
decoration-muted
decoration-primary
decoration-success
decoration-warning

- *Attributes and options - List*
  - El atributo ***decoration-{$name}*** permite colorear las filas (doc)
- Queremos para el *tree view* de estate.property.offer
  - Ocultar el estado
  - Mostrar las aceptadas en verde y las rechazadas en rojo

```
<tree string="Property Offers" ...
    decoration-success="state == 'accepted'"
    decoration-danger="state == 'refused'">
```

- Queremos para el *tree view* de estate.property
  - Mostrar si oferta recibida→verde, aceptada→verde+negrita, vendida→*muted*

```
<tree string="Properties"
    decoration-success="state in ('offer_received', 'offer_accepted')"
    decoration-muted="state == 'sold'"
    decoration-bf="state == 'offer_accepted'"
```

- Actualiza los modelos *offer* y *property*
  - ✓ Añade a las *tree views* los atributos adecuados

Capítulo 12 – Sprinkles (10/15)

Ejemplo: *action* y filtro

- *Attributes and options - Search* (doc[1] y doc[2])
  - ➢ Podemos indicar qué filtro(s) queremos que esté activo por defecto a nivel de *action* usando el ***search_default_{$name} action context***, donde {$name} es el nombre del filtro.
- Queremos para estate.property
  - ➢ Activar por defecto el filtro 'available'

```
<field name="context">{'search_default_available': 1}</field>
```

- Actualiza el *action* de estate.*property*
  - ✓ Añade el campo *context* correctamente configurado

Capítulo 12 – Sprinkles (11/15)

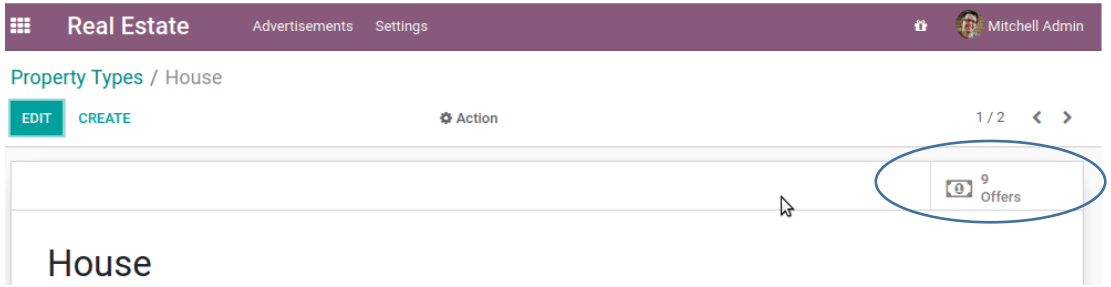
- *Attributes and options - Search* (doc[1] y doc[2])
  - El atributo ***filter_domain*** de los *field* de una *search view* permite sobreescribir lo que indique el *domain* para buscar por ese campo
- Queremos para la *search view* de estate.property
  - al buscar por 'living área' seleccione propiedades de "al menos" el valor introducido (sería raro buscar el valor exacto)

```
<field name="living_area" filter_domain="[('living_area', '>=', self)]"/>
```

- Actualiza el *search view* de estate.*property*
  - ✓ Añade el atributo *filter_domain* correctamente configurado

Capítulo 12 – Sprinkles (12/15)

- *Stat buttons*
  - Botones para acceder a datos relacionados



  - Usaremos '*related fields*' (doc), como si fuera un caso específico de un *computed field*:

```
partner_id = fields.Many2one("res.partner", string="Partner")
description = fields.Char(related="partner_id.name")
```

equivale a:

```
partner_id = fields.Many2one("res.partner", string="Partner")
description = fields.Char(compute="_compute_description")

@api.depends("partner_id.name")
def _compute_description(self):
    for record in self:
        record.description = record.partner_id.name
```

Si cambia el *partner_id.name* cambia el *description*

Capítulo 12 – Sprinkles (13/15)

- *Stat buttons (cont.)*
- Queremos para el form view de estate.property.type
  - ➢ Un *stat button* que muestre las ofertas relacionadas con las propiedades de ese tipo
- Actualiza el módulo (ofertas y tipos) [1/3]
  - ✓ Crea el campo relacionado

Offer

```
property_type_id = fields.Many2one("estate.property.type",
    related="property_id.property_type_id", string="Property Type", store=True)
```

Type

```
offer_ids = fields.Many2many("estate.property.offer",
    string="Offers", compute="_compute_offer")
offer_count = fields.Integer(string="Offers Count", compute="_compute_offer")
```

Capítulo 12 – Sprinkles (14/15)

- *Stat buttons (cont.)*
- Actualiza el módulo (ofertas y tipos) [2/3]
  - ✓Calcula el número de ofertas

Type

```
offer_count = fields.Integer(string="Offers Count", compute="_compute_offer")
```

```
def _compute_offer(self):
    # This solution is quite complex (vs. search in a loop)
    data = self.env["estate.property.offer"].read_group(
        [("property_id.state", "!=", "canceled"),
         ("property_type_id", "!=", False)],
        ["ids:array_agg(id)", "property_type_id"],
        ["property_type_id"],
    )
    mapped_count = {d["property_type_id"][0]: d["property_type_id_count"] for d in data}
    mapped_ids = {d["property_type_id"][0]: d["ids"] for d in data}
    for prop_type in self:
        prop_type.offer_count = mapped_count.get(prop_type.id, 0)
        prop_type.offer_ids = mapped_ids.get(prop_type.id, [])
```

Capítulo 12 – Sprinkles (15/15)

- *Stat buttons (cont.)*
- Actualiza el módulo (ofertas y tipos) [3/3]
  - ✓Añade el stat button al estate_property_type_view_form

```
<form><sheet>
    <div class="oe_button_box" name="button_box">
        <button name="%(estate.estate_property_offer_action)d"
                type="action" class="oe_stat_button" icon="fa-money">
            <field name="offer_count" widget="statinfo" string="Offers"/>
        </button>
```

  - ✓Añade un dominio al estate.property.offer.action para filtrar

```
<record id="estate_property_offer_action" model="ir.actions.act_window">
    <field name="name">Property Offers</field>
    <field name="res_model">estate.property.offer</field>
    <field name="domain">[('property_type_id','=', active_id)]</field>
    <field name="view_mode">tree,form</field>
</record>
```

Registro actual (ejemplo)

y más…

https://www.odoo.com/documentation/master/developer/tutorials/getting_started.html

- Chapter 1: Architecture Overview
- Chapter 2: Development environment setup
- Chapter 3: A New Application
- Chapter 4: Models And Basic Fields
- Chapter 5: Security - A Brief Introduction
- Chapter 6: Finally, Some UI To Play With
- Chapter 7: Basic Views
- Chapter 8: Relations Between Models
- Chapter 9: Computed Fields And Onchanges
- Chapter 10: Ready For Some Action?
- Chapter 11: Constraints
- Chapter 12: Add The Sprinkles
- Chapter 13: Inheritance
- Chapter 14: Interact With Other Modules
- Chapter 15: A Brief History Of QWeb
- Chapter 16: The final word